



Sichuan University -
Pittsburgh Institute

ECE0202 Embedded Processors & Interfacing + Lab

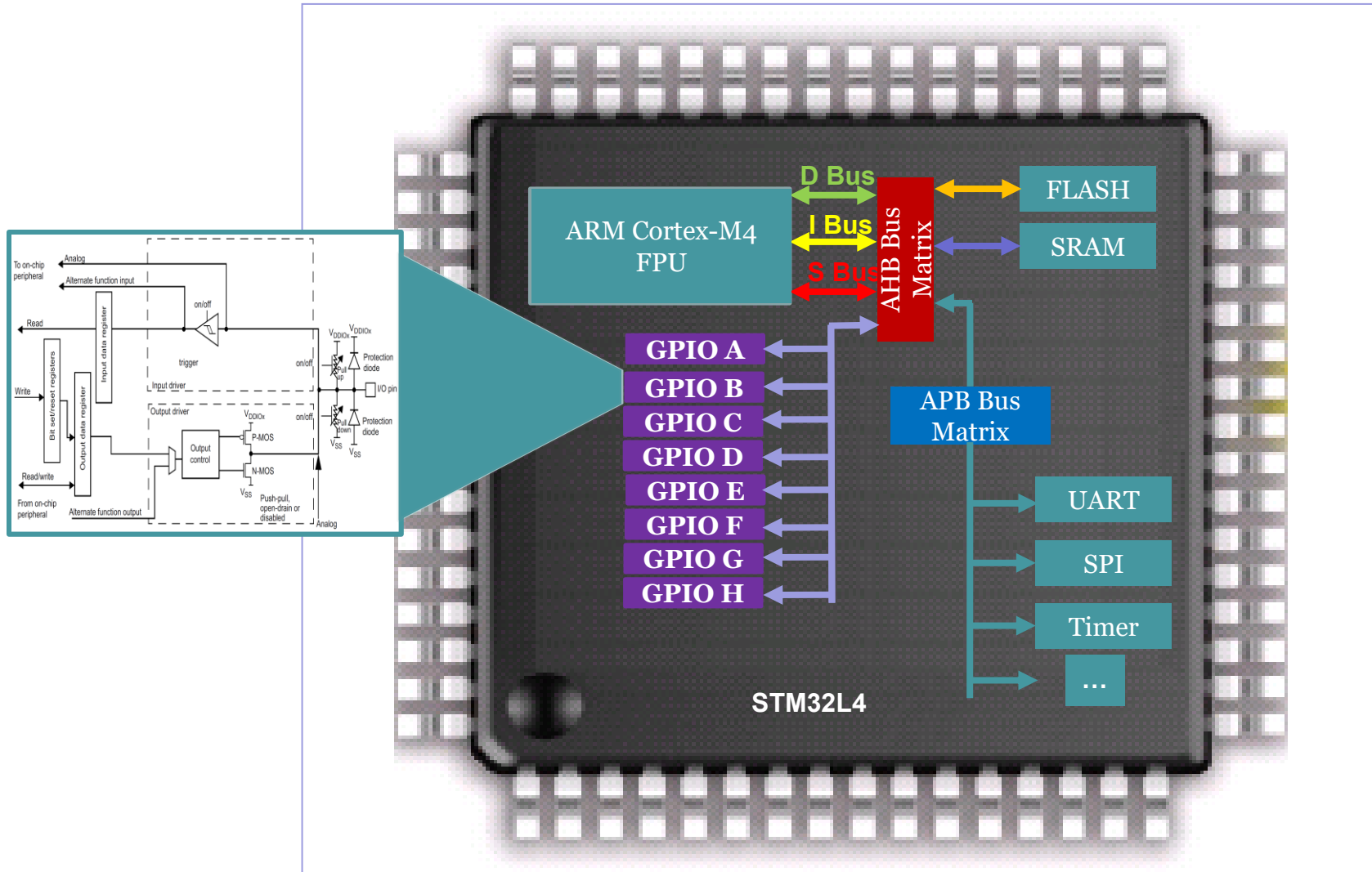
Sichuan University-Pittsburgh Institute



Course Objective

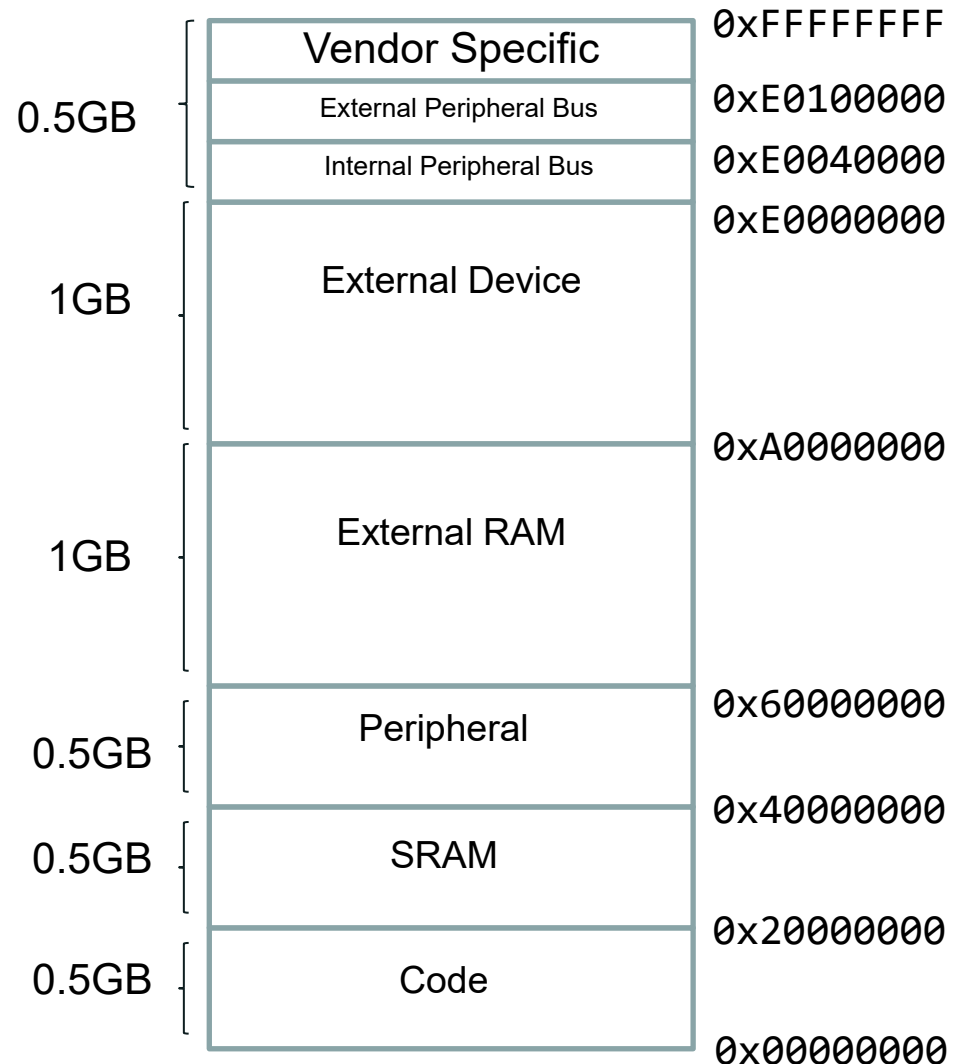
- How is data organized in memory?
 - Big Endian vs Little Endian
- How is data addressed?
 - Pre-index
 - Post-index
 - Pre-index with update

I/O Peripherals

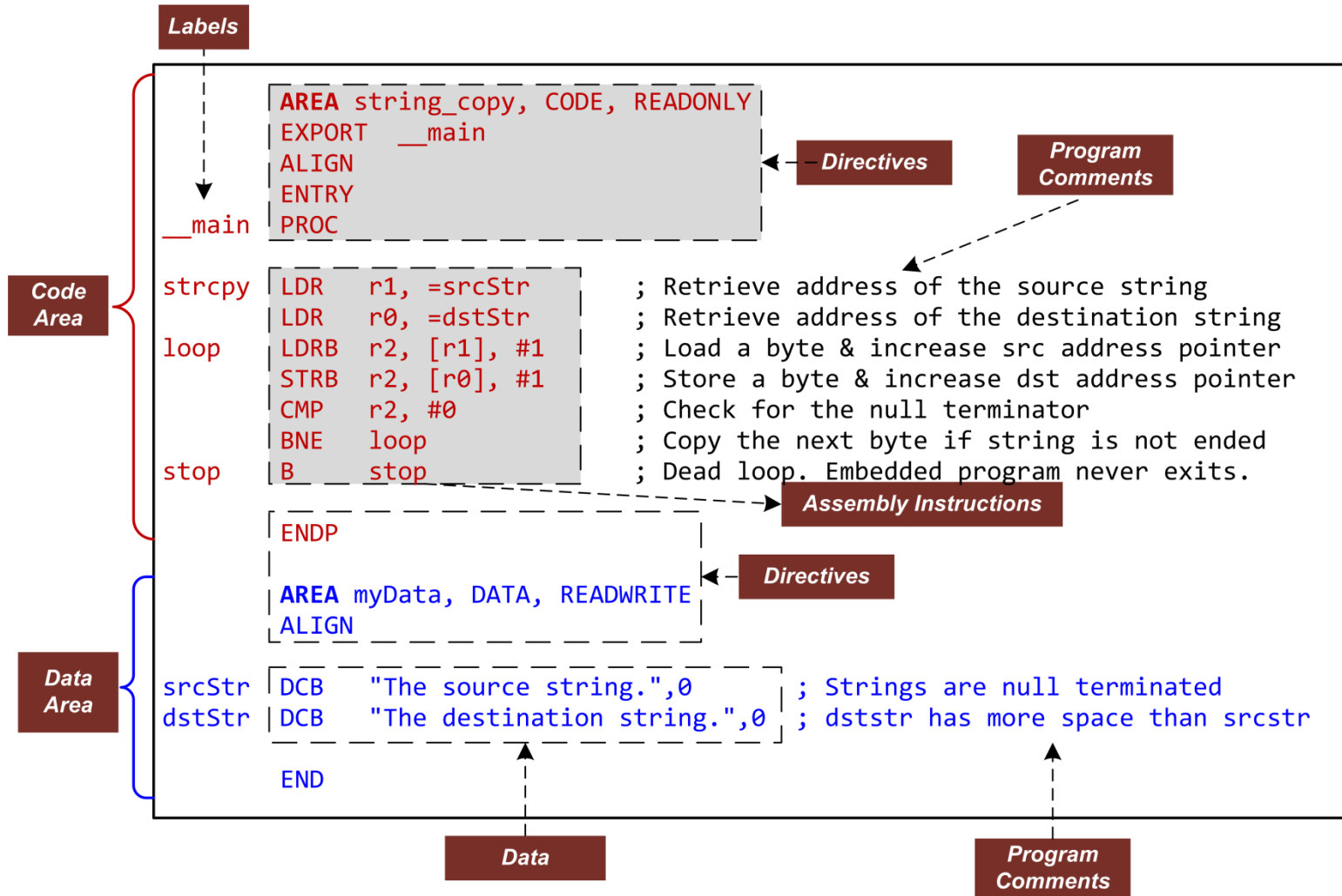


Cortex-M3 & Cortex-M4 Memory Map

- 32-bit Memory Address
- 2^{32} bytes of memory space (4 GB)
- Harvard architecture: physically separated instruction memory and data memory

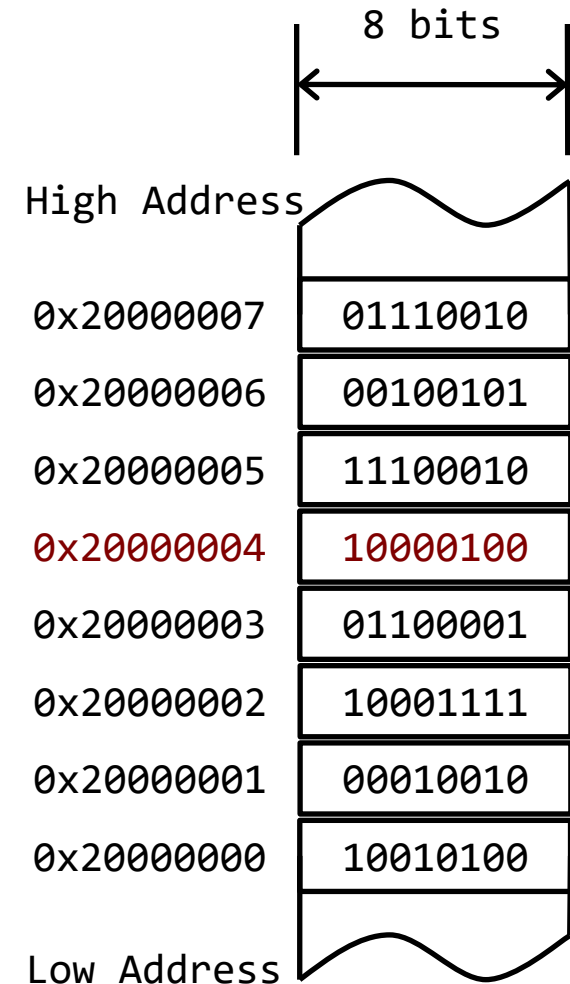


First Assembly



Logic View of Memory

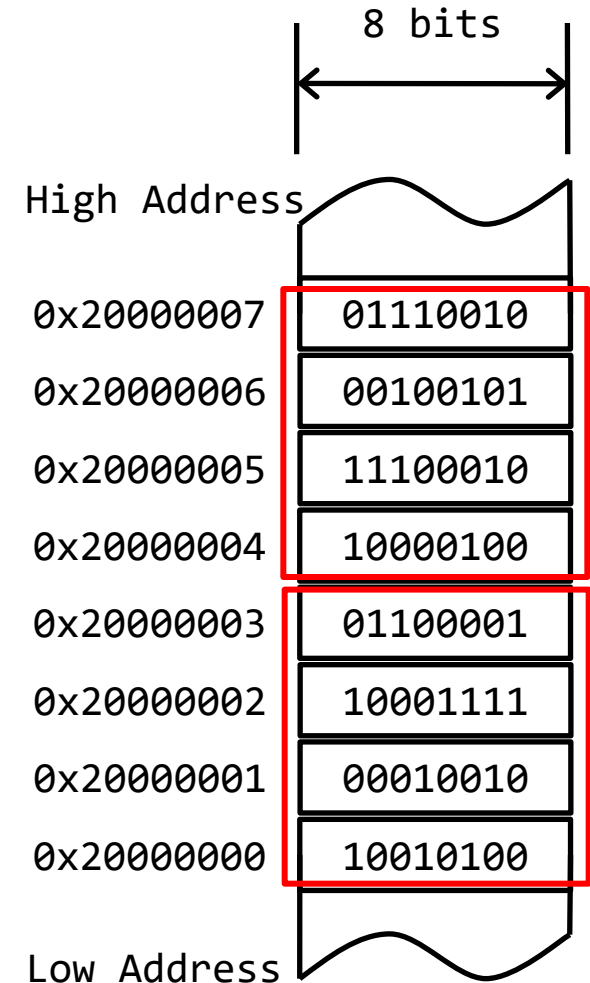
- By grouping bits together we can store more values
 - 8 bits = 1 **byte**
 - 16 bits = 2 bytes = 1 **halfword**
 - 32 bits = 4 bytes = 1 **word**
 - From software perspective, **memory is an addressable array of bytes**.
 - The byte stored at the memory address `0x20000004` is `0b10000100`
- `0b10000100` → `0x84` → 132
 Binary Hexadecimal Decimal



Logic View of Memory

- When we refer to memory locations by address, we can only do so in units of bytes, halfwords or words
- Words
 - 32 bits = 4 bytes = 1 word = 2 halfwords
 - In the right diagram, we have two words at addresses:
 - 0x20000000
 - 0x20000004
 - Can you store a word anywhere? **NO.**
 - A word can only be stored at an address that's divisible by 4.
 - Memory address of a word is the lowest address of all four bytes in that word.

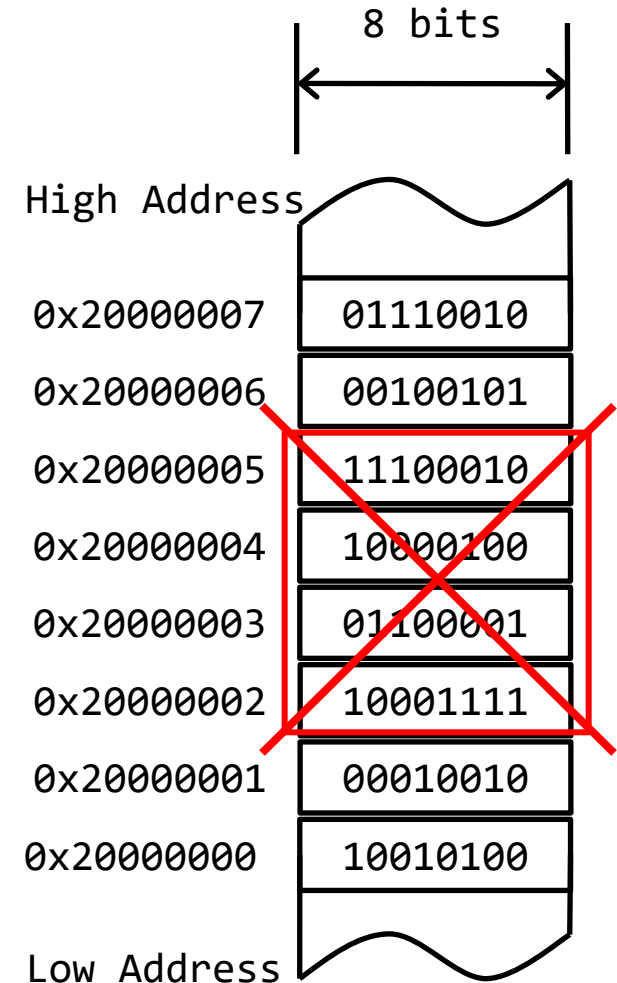
$$\text{Word-address mod } 4 = 0$$



Logic View of Memory

- When we refer to memory locations by address, we can only do so in units of bytes, halfwords or words
- Words
 - 32 bits = 4 bytes = 1 word = 2 halfwords
 - In the right diagram, we have two words at addresses:
 - 0x20000000
 - 0x20000004
 - Can you store a word anywhere? **NO.**
 - A word can only be stored at an address that's divisible by 4.
 - Memory address of a word is the lowest address of all four bytes in that word.

We cannot store a word at address 0x20000002.

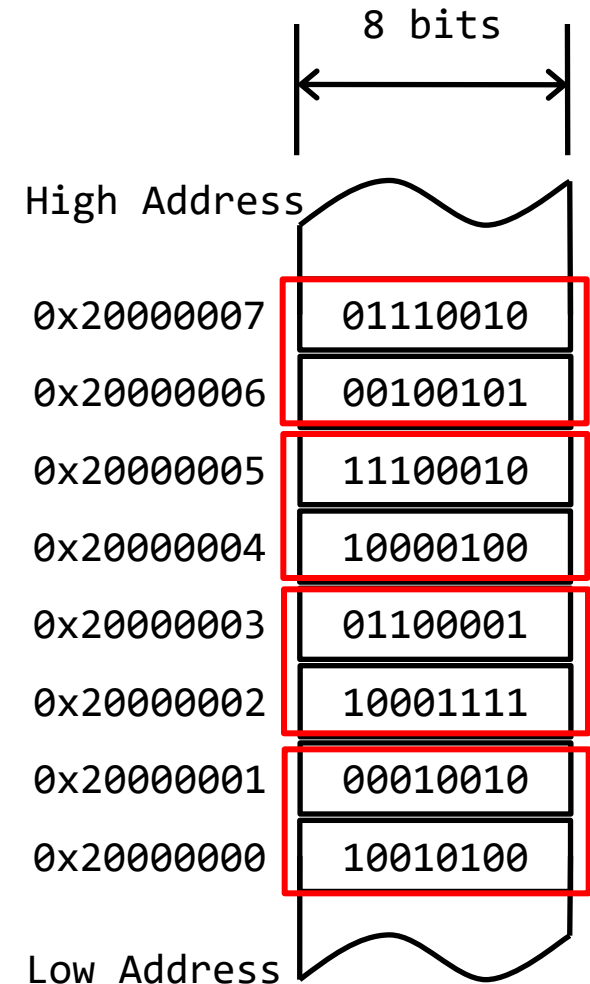


$$\text{Word-address mod } 4 = 0$$

Logic View of Memory

- When we refer to memory locations by address, we can only do so in units of bytes, halfwords or words
- Halfwords
 - **16 bits = 2 bytes = 1 halfword**
 - The right diagram has four halfwords at addresses of:
 - 0x20000000
 - 0x20000002
 - 0x20000004
 - 0x20000006
 - Can you store a halfword anywhere? **NO.**
 - A halfword can only be stored at an address that's divisible by 2.
 - Memory address of a halfword is the lowest address of its two bytes.

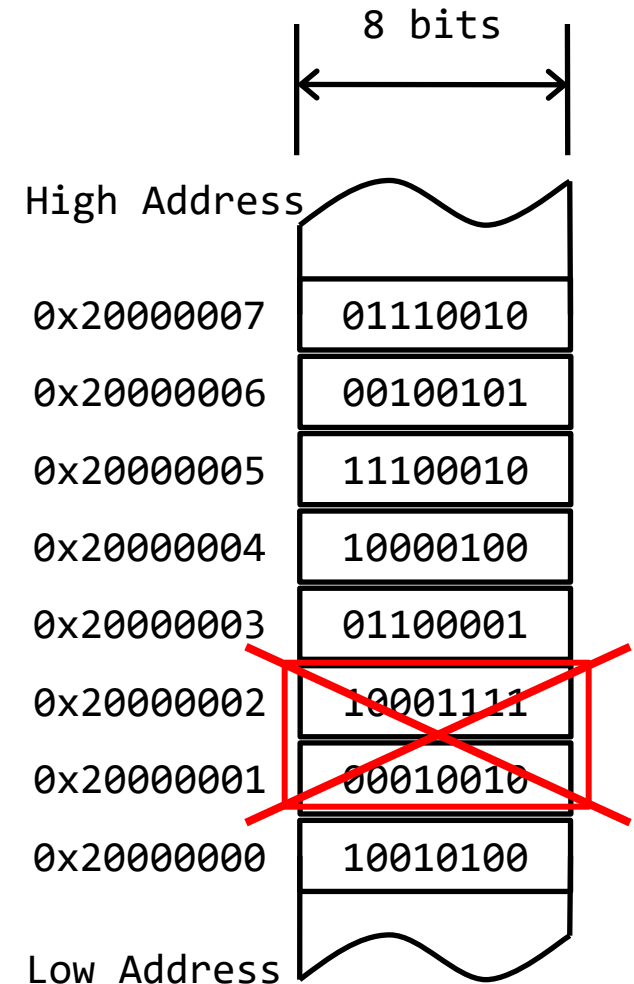
$$\text{Halfword-address mod } 2 = 0$$



Logic View of Memory

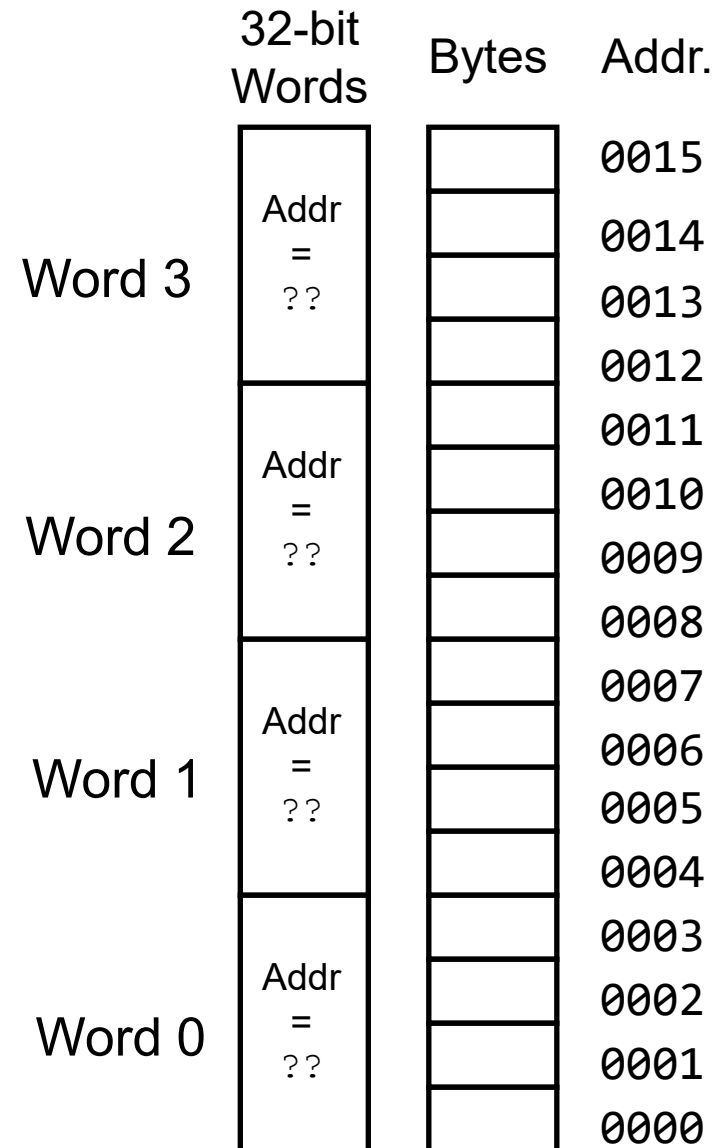
- When we refer to memory locations by address, we can only do so in units of bytes, halfwords or words
- Halfwords
 - 16 bits = 2 bytes = 1 halfword
 - The right diagram has four halfwords at addresses of:
 - 0x20000000
 - 0x20000002
 - 0x20000004
 - 0x20000006
 - Can you store a halfword anywhere? **NO.**
 - A halfword can only be stored at an address that's divisible by 2.
 - Memory address of a halfword is the lowest address of its two bytes.

We cannot store a halfword at address 0x20000001.



$$\text{Halfword-address mod } 2 = 0$$

What are the
memory address of
these four words?



Answer

What are the
memory address of
these four words?

	32-bit Words	Bytes	Addr.
Word 3	Addr = 0x0012		0015
			0014
			0013
			0012
Word 2	Addr = 0x0008		0011
			0010
			0009
			0008
Word 1	Addr = 0x0004		0007
			0006
			0005
			0004
Word 0	Addr = 0x0000		0003
			0002
			0001
			0000

endianness

High address

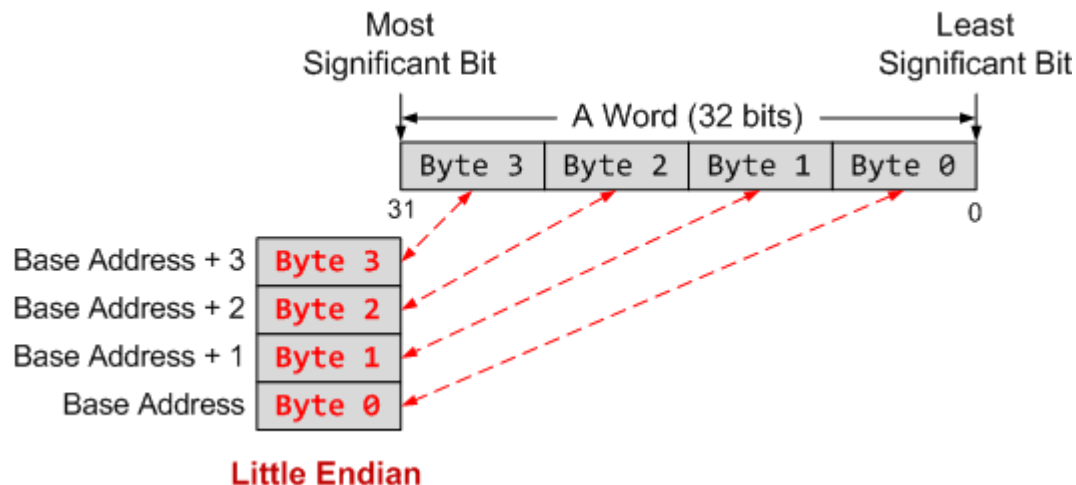


Low address



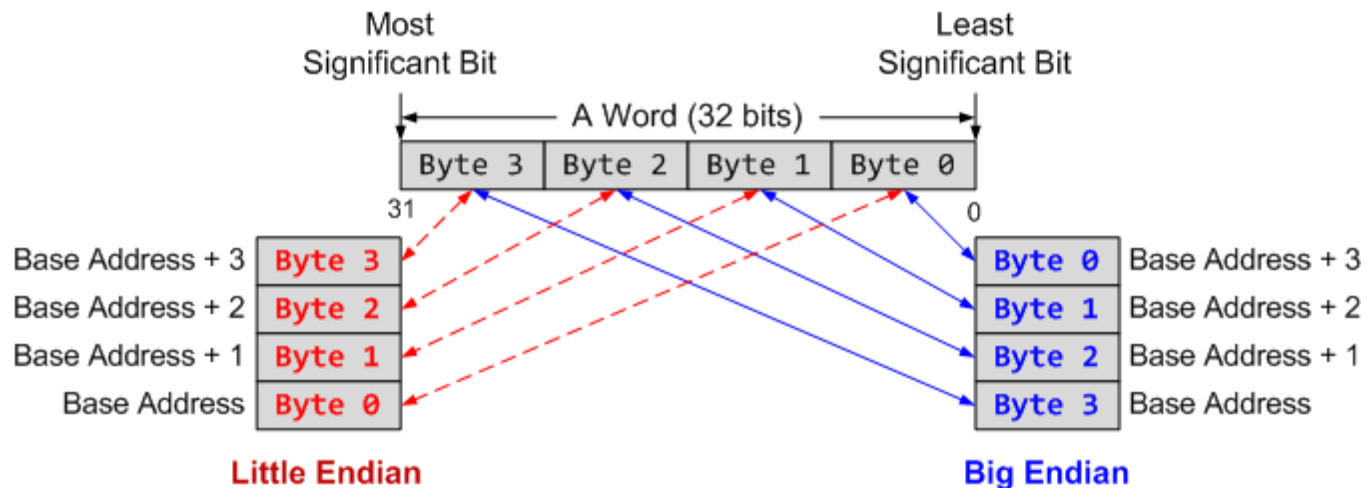
endianness

- **Little Endian:** Most significant byte is stored at a high address
- **Big Endian:** Most significant byte is stored at a low address
- Regardless endian, the address of a word is defined as the lowest address of all bytes it occupies.
- ARM is *Little Endian by default*. However it can be made Big Endian by configuration.



endianness

- **Little Endian:** Most significant byte is stored at a high address
- **Big Endian:** Most significant byte is stored at a low address
- Regardless endian, the address of a word is defined as the lowest address of all bytes it occupies.
- ARM is *Little Endian by default*. However it can be made Big Endian by configuration.



Example

If big endianness is used

The word stored at address
0x20008000 is

0xEE8C90A7

Memory Address	Memory Data
0x20008003	0xA7
0x20008002	0x90
0x20008001	0x8C
0x20008000	0xEE

Example

If little endianness is used

The word stored at address
0x20008000 is

0xA7908CEE

Endian only specifies byte order, not bit
order in a byte!

Memory Address	Memory Data
0x20008003	0xA7
0x20008002	0x90
0x20008001	0x8C
0x20008000	0xEE

Data Addressing using LOAD/STORE

- Pre-index
- Post-index
- Pre-index with update

Load-Modify-Store

C statement

```
X = X + 1;
```



; Assume the memory address of x is stored in r1

```
LDR r0, [r1]      ; load value of x from memory
```

```
ADD r0, r0, #1     ; x = x + 1
```

```
STR r0, [r1]       ; store x into memory
```

Load Instructions

- **LDR rt, [rs]**
 - fetch data from memory into register rt.
 - The memory address is specified in register rs.
 - For Example:

; Assume r0 = 0x08200004

; Load a word:

LDR r1, [r0] ; r1 = Memory.word[0x08200004]

Store Instructions

- **STR rt, [rs]**
 - save data in register rt into memory
 - The memory address is specified in a base register rs.
 - For Example:

; Assume r0 = 0x08200004
; Store a word
STR r1, [r0] ; Memory.word[0x08200004] = r1

Single register data transfer

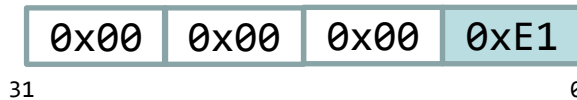
LDR	Load Word
LDRB	Load Byte
LDRH	Load Halfword
LDRSB	Load Signed Byte
LDRSH	Load Signed Halfword

STR	Store Word
STRB	Store Lower Byte
STRH	Store Lower Halfword

Load a Byte, Half-word, Word

Load a Byte

LDRB r1, [r0]



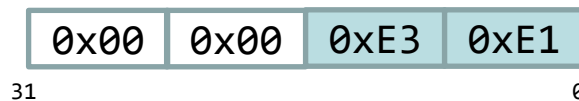
0x02000003	0x87
0x02000002	0x65
0x02000001	0xE3
0x02000000	0xE1

Little Endian

Assume
r0 = 0x02000000

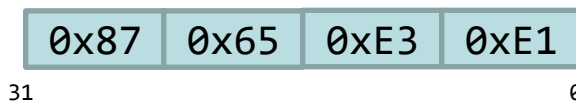
Load a Halfword

LDRH r1, [r0]



Load a Word

LDR r1, [r0]



Sign Extension

Load a Signed Byte

LDRSB r1, [r0]



Load a Signed Halfword

LDRSH r1, [r0]



0x20000003	0x87
0x20000002	0x65
0x20000001	0xE3
0x20000000	0xE1

Little Endian

Assume
r0 = 0x02000000

Facilitate subsequent 32-bit signed arithmetic!

Address

- Address accessed by LDR/STR is specified by a base register **plus an offset**
- For word and unsigned byte accesses, offset can be
 - An unsigned 12-bit immediate value (i.e. 0 - 4095 bytes).
`LDR r0, [r1, #8]`
 - A register, optionally shifted by an immediate value
`LDR r0, [r1, r2]`
`LDR r0, [r1, r2, LSL#2]`
- This can be either added or subtracted from the base register:
`LDR r0, [r1, #-8]`
`LDR r0, [r1, -r2]`
`LDR r0, [r1, -r2, LSL#2]`
- For halfword and signed halfword / byte, offset can be:
 - An unsigned 8 bit immediate value (i.e. 0-255 bytes).
 - A register (unshifted).
- Choice of *pre-indexed* or *post-indexed* addressing

Pre-index

Pre-Index: LDR r1, [r0, #4]

Offset = 4

r0 = 0x20008000

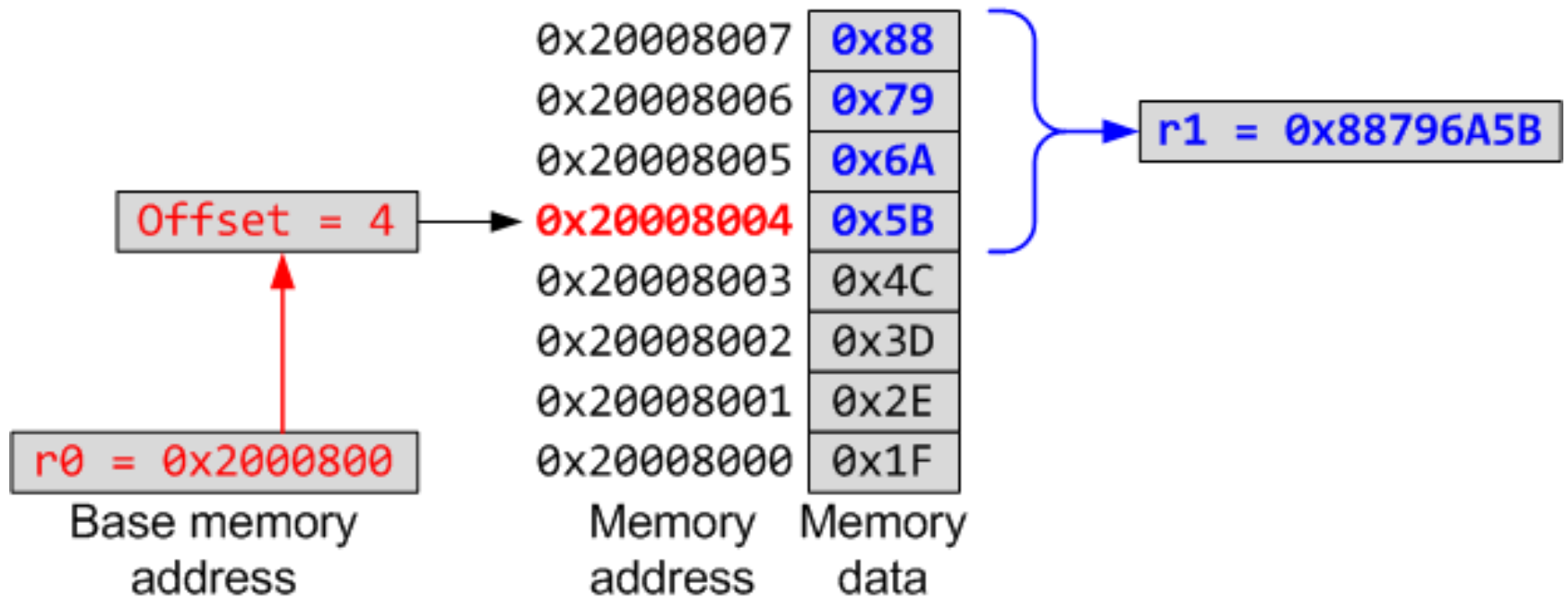
Base memory
address

0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F
Memory address	Memory data

r1

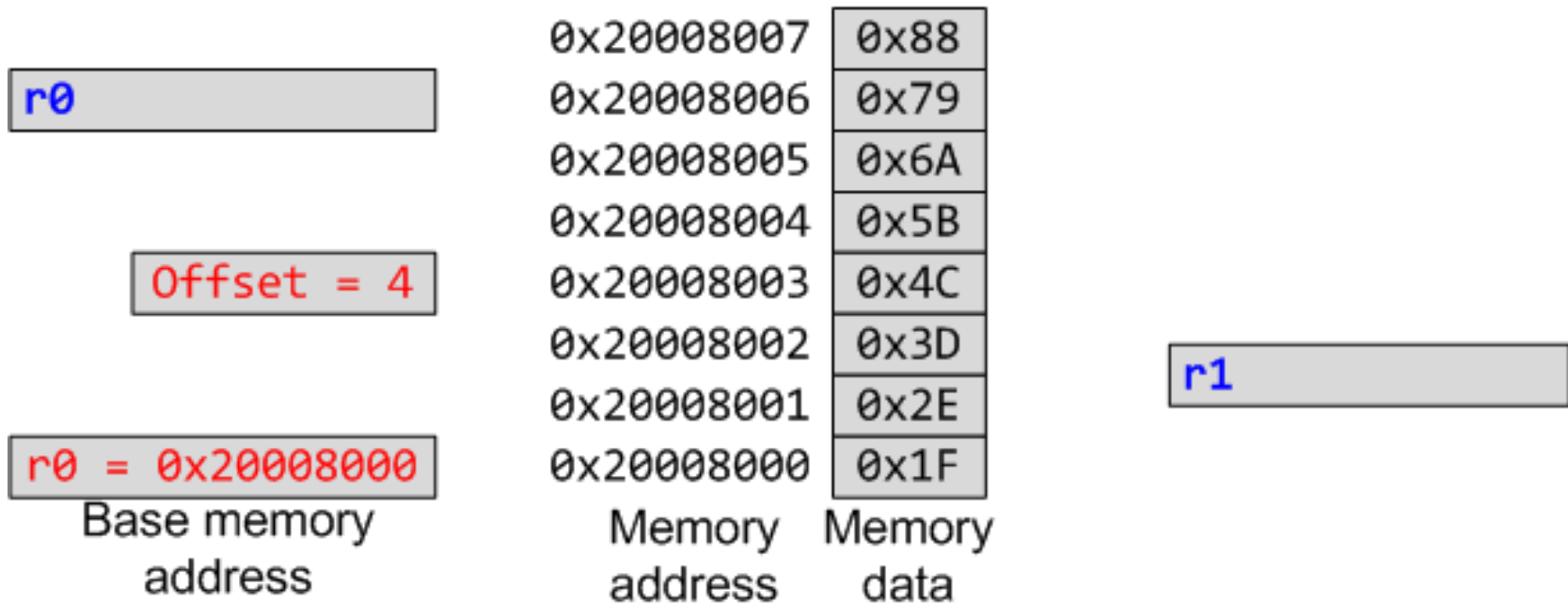
Pre-index

Pre-Index: LDR r1, [r0, #4]



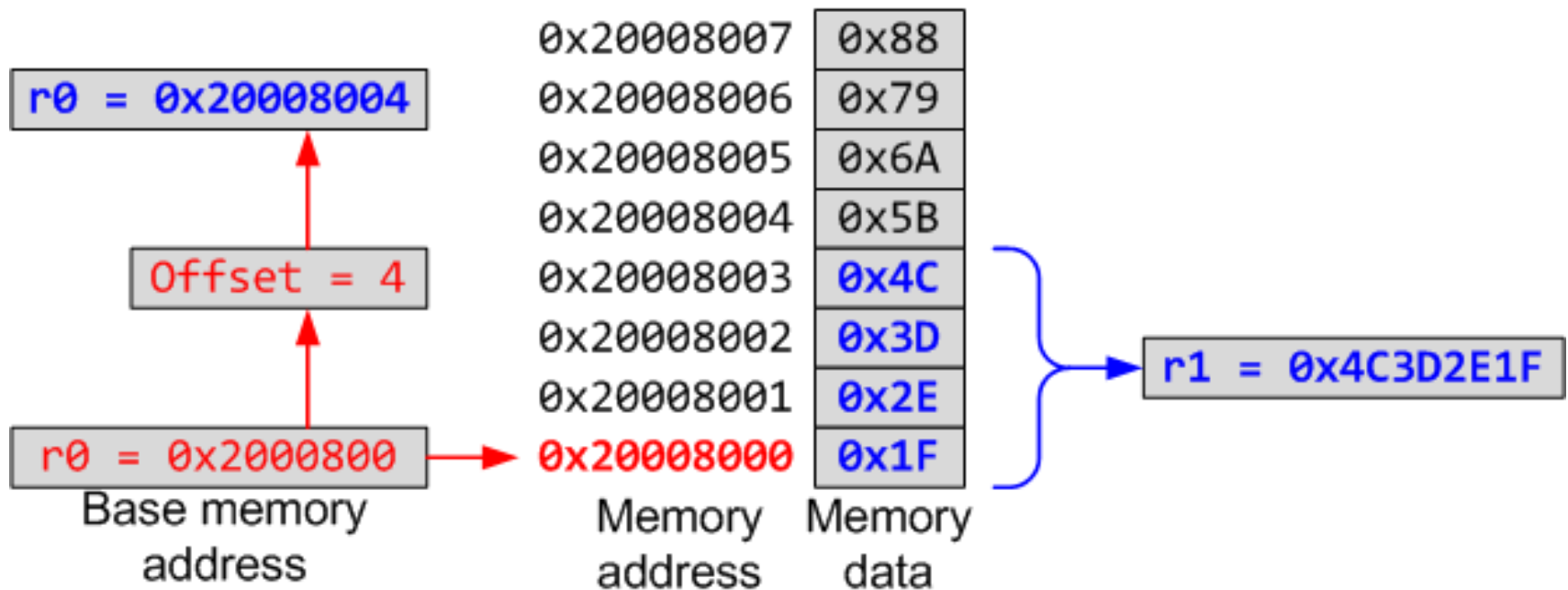
Post-index

Post-Index: LDR r1, [r0], #4



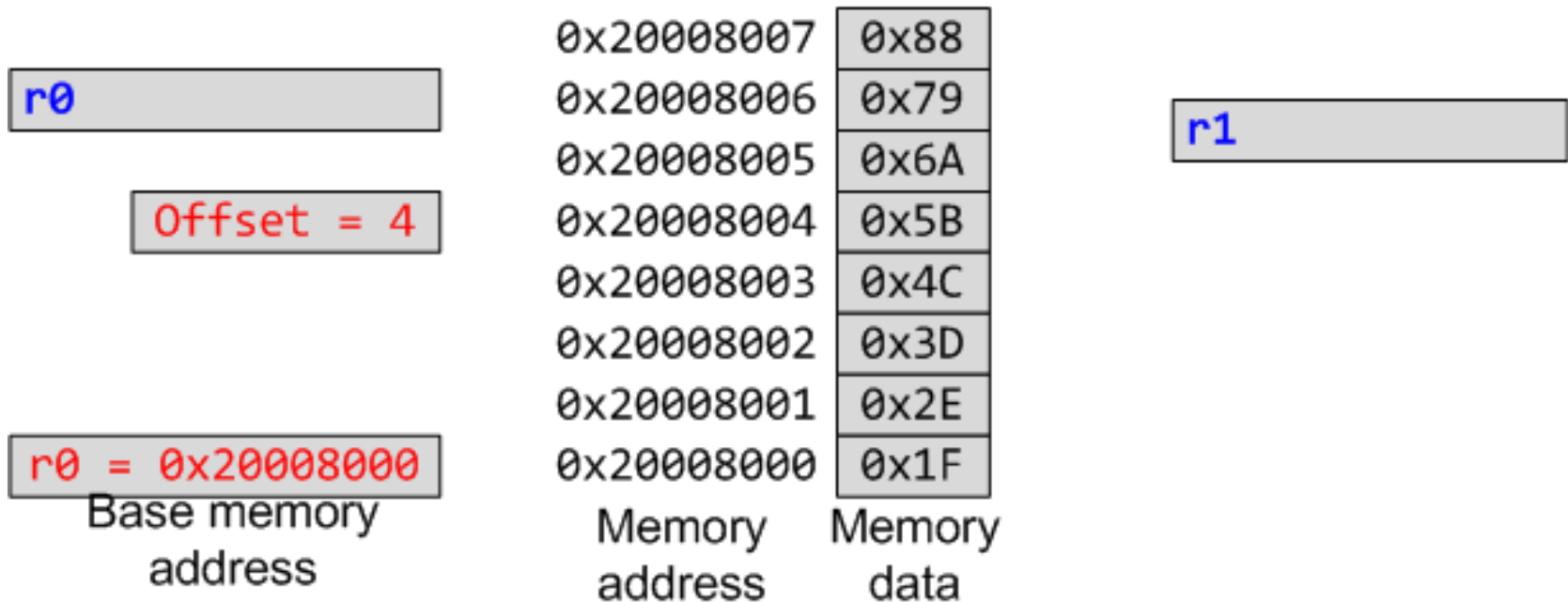
Post-index

Post-Index: LDR r1, [r0], #4



Pre-index with Updates

Pre-Index with Update: `LDR r1, [r0, #4]!`



Pre-index with Updates

Pre-Index with Update: `LDR r1, [r0, #4]!`



Summary of Pre-index and Post-index

Index Format	Example	Equivalent
Pre-index	LDR r1, [r0, #4]	$r1 \leftarrow \text{memory}[r0 + 4]$, r0 is unchanged
Pre-index with update	LDR r1, [r0, #4]!	$r1 \leftarrow \text{memory}[r0 + 4]$ $r0 \leftarrow r0 + 4$
Post-index	LDR r1, [r0], #4	$r1 \leftarrow \text{memory}[r0]$ $r0 \leftarrow r0 + 4$

Example

LDRH r1, [r0]

; r0 = 0x20008000

r1 before load

0x12345678

r1 after load

0x0000CDEF

Memory Address	Memory Data
0x20008003	0x89
0x20008002	0xAB
0x20008001	0xCD
0x20008000	0xEF

Example

LDSB r1, [r0]

; r0 = 0x20008000

r1 before load

0x12345678

r1 after load

0xFFFFFFFF

Memory Address	Memory Data
0x20008003	0x89
0x20008002	0xAB
0x20008001	0xCD
0x20008000	0xEF

Example

STR r1, [r0], #4

; r0 = 0x20008000, r1=0x76543210

r0 before store

0x20008000

r0 after store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

STR r1, [r0], #4

; r0 = 0x20008000, r1=0x76543210

r0 before store

0x20008000

r0 after store

0x20008004

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x76
0x20008002	0x54
0x20008001	0x32
0x20008000	0x10

Example

STR r1, [r0, #4]

; r0 = 0x20008000, r1=0x76543210

r0 before the store

0x20008000

r0 after the store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

STR r1, [r0, #4]

; r0 = 0x20008000, r1=0x76543210

r0 before store

0x20008000

r0 after store

0x20008000

Memory Address	Memory Data
0x20008007	0x76
0x20008006	0x54
0x20008005	0x32
0x20008004	0x10
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

STR r1, [r0, #4]!

; r0 = 0x20008000, r1=0x76543210

r0 before store

0x20008000

r0 after store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

STR r1, [r0, #4]!

; r0 = 0x20008000, r1=0x76543210

r0 before store

0x20008000

r0 after store

0x20008004

Memory Address	Memory Data
0x20008007	0x76
0x20008006	0x54
0x20008005	0x32
0x20008004	0x10
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

If big endianness is used

r11 before load

0x12345678

r11 after load

0xA7908CEE

```
LDR r11, [r0]  
; r0 = 0x20008000
```

Memory Address	Memory Data
0x20008003	0xEE
0x20008002	0x8C
0x20008001	0x90
0x20008000	0xA7

Load/Store Multiple Registers

`STMxx rn{!}, {register_list}`

`LDMxx rn{!}, {register_list}`

- `xx` = IA, IB, DA, or DB

Addressing Modes	Description	Instructions
IA	Increment After	STMIA, LDMIA
IB	Increment Before	STMIB, LDMIB
DA	Decrement After	STMDA, LDMDA
DB	Decrement Before	STMDB, LDMDB

- **IA**: address is incremented by 4 after a word is loaded or stored.
- **IB**: address is incremented by 4 before a word is loaded or stored.
- **DA**: address is decremented by 4 after a word is loaded or stored.
- **DB**: address is decremented by 4 before a word is loaded or stored.

Load/Store Multiple Registers

- The order in which registers are listed does not matter
 - For STM/LDM, the lowest-numbered register is stored/loaded at the lowest memory address.

Store Multiple Registers

STMxx r0!, {r3,r1,r7,r2}

STMIA

Increment After

STMIB

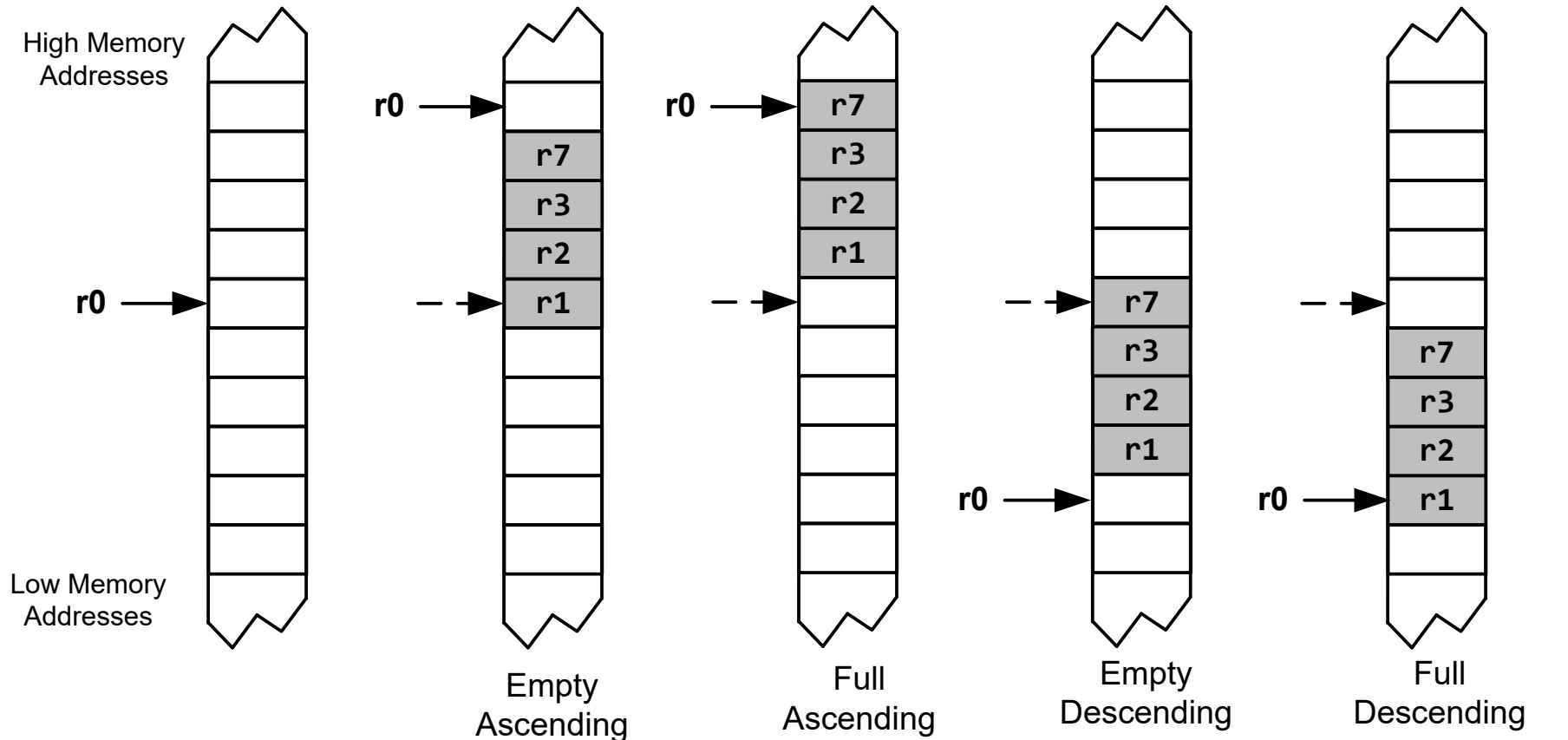
Increment Before

STMDA

Decrement After

STMDB

Decrement Before



Load Multiple Registers

LDMxx r0!, {r3,r1,r7,r2}

LDMIA

Increment After

LDMIB

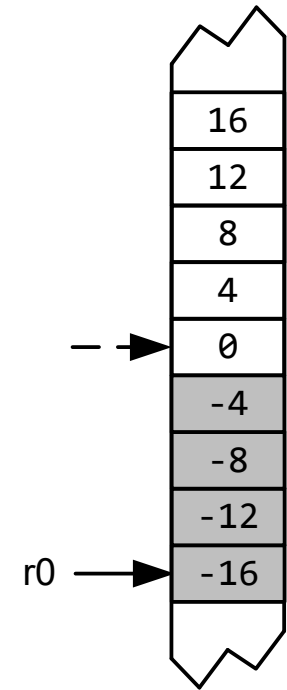
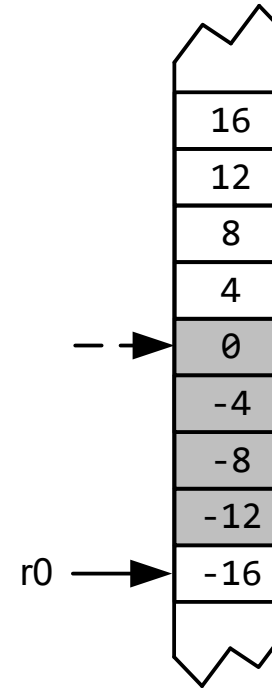
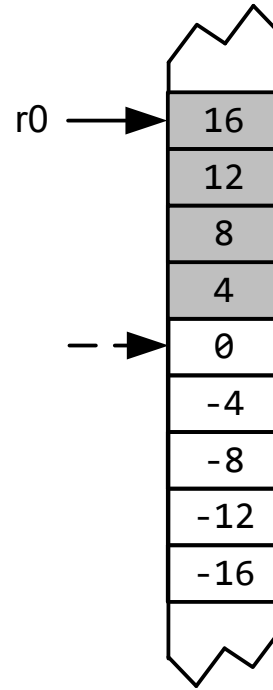
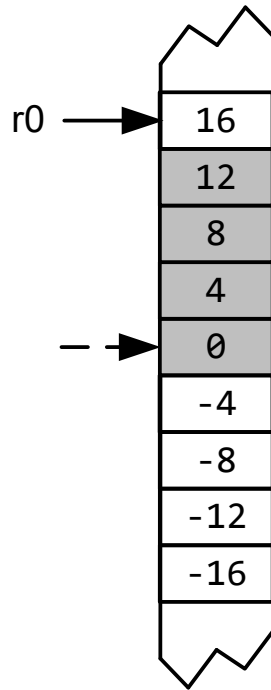
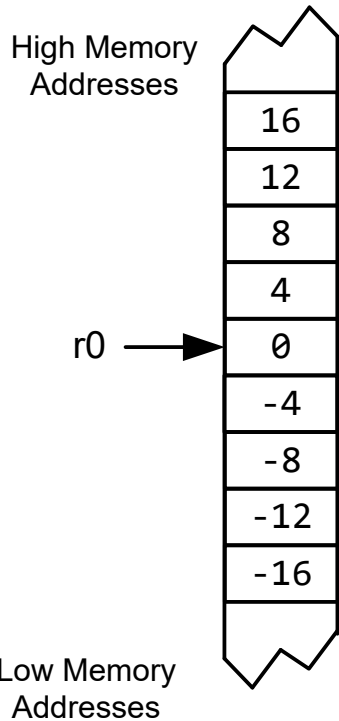
Increment Before

LDMDA

Decrement After

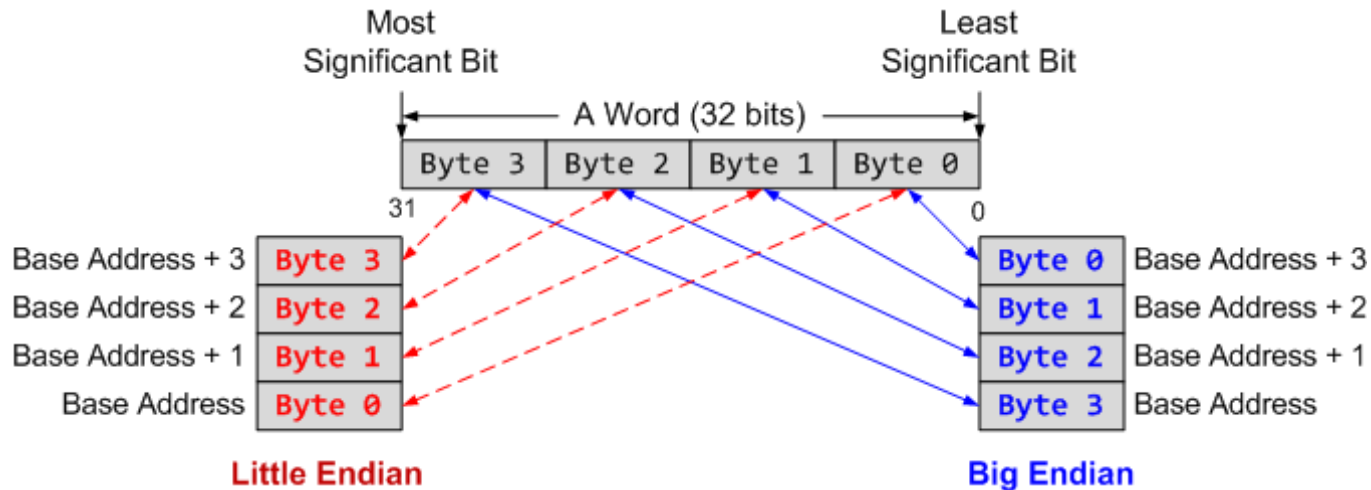
LDMDB

Decrement Before



Summary

- Memory address is always in terms of bytes.
- How data is organized in memory?



- How data is addressed?

Addressing Format	Example	Equivalent
Pre-index	LDR r1, [ro, #4]	$r1 \leftarrow \text{memory}[ro + 4]$, ro is unchanged
Pre-index with update	LDR r1, [ro, #4]!	$r1 \leftarrow \text{memory}[ro + 4]$ $ro \leftarrow ro + 4$
Post-Index	LDR r1, [ro], #4	$r1 \leftarrow \text{memory}[ro]$ $ro \leftarrow ro + 4$

Addressing GPIO in Assembly

- The GPIO module has a base address
- The GPIO registers are specified as offsets from the base address
- You should use pre-indexing to access GPIO registers

Addressing GPIO in Assembly

- Specify Base Address
 - GPIOA_BASE EQU 0x48000000
- Specify Register Offsets
 - MODER EQU 0x0 (no offset)
 - MODER EQU 0x4
 - IDR EQU 0x10
 - ODR EQU 0x14

Load/modify/store Logic

1. Load base address into a core register
2. Load data from a register into another core register by using a load with pre-index instruction
3. Modify data (cleat/set/toggle)
4. Store back into memory by using a store with pre-index instruction

Load/modify/store example

GPIOA_BASE EQU 0x48000000 ; base address for GPIOA

MODER EQU 0x0 ; Offset for MODER

LDR r0, =GPIOA_BASE ;step 1 – load base address into a core register

LDR r1, [r0, #MODER] ;step 2 – load data from MODER into r1

BIC r1, #0x3 ;step 3 - modify data (in this case clearing bits 0 & 1)

STR r1, [r0, #MODER] ;step 4 – store data back into MODER memory location